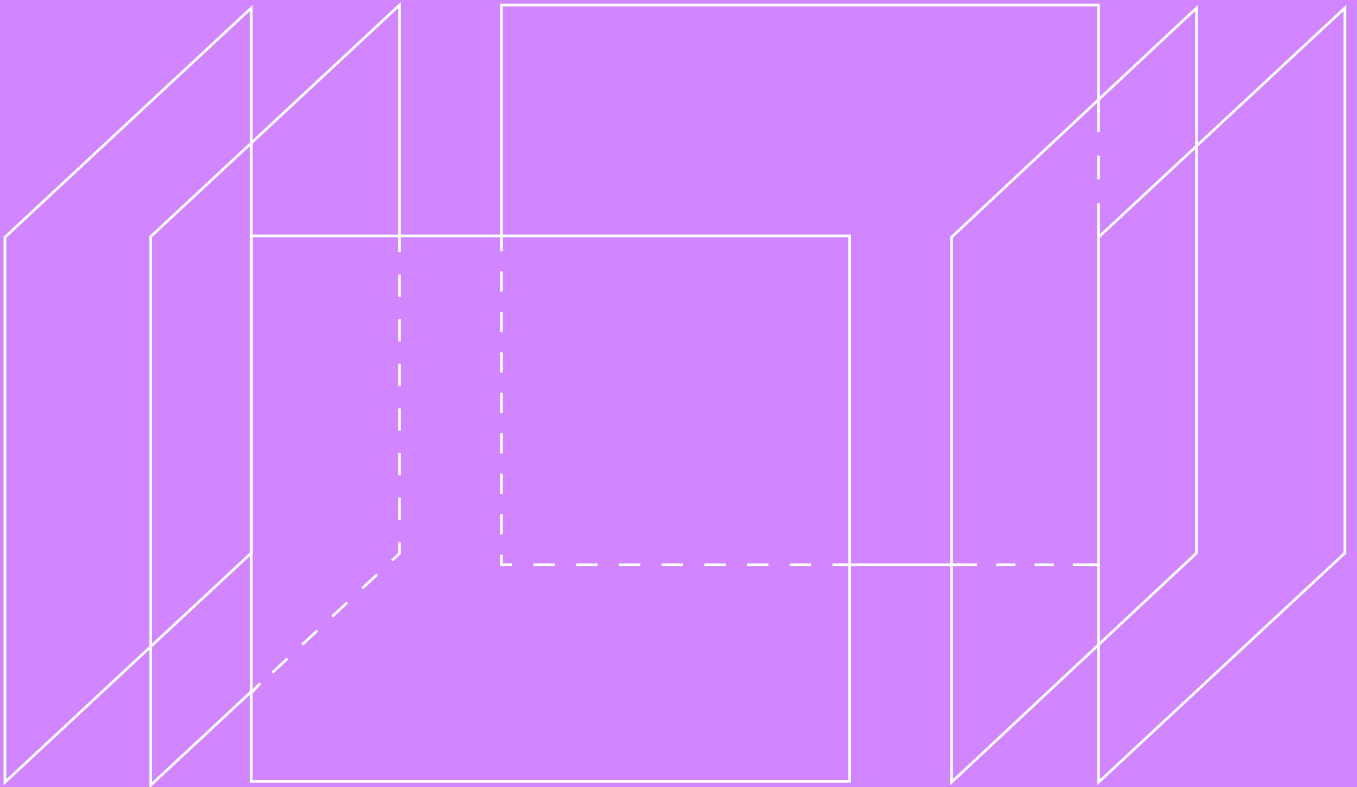


Транзакции. Уровни изоляции транзакций



SQL И БАЗЫ ДАННЫХ (ДЛЯ РАЗРАБОТЧИКОВ)

ЛОНГРИД

НЕДЕЛЯ 11

Проектирование высоконагруженных баз данных начинается с осознания одного фундаментального факта: СУБД постоянно врёт разработчику. Это враньё заложено в самом контракте ACID, который каждая реляционная база заключает с твоим кодом.

Атомарность (Atomicity), **согласованность** (Consistency) и **долговечность** (Durability) — это строгие, бинарные гарантии. База либо умеет надёжно писать данные на диск и откатывать ошибки, либо она не имеет права называться базой данных. Но вот буква «I» — **изоляция** (Isolation) — это не константа. Это грандиозная архитектурная иллюзия, за поддержание которой система платит колоссальную цену.

СУБД обещает, что твоя транзакция работает с данными в гордом одиночестве, словно в системе нет других пользователей. На практике же в соседних потоках памяти с теми же самыми строками прямо сейчас работают тысячи других транзакций. **Изоляция** — это всегда спектр компромиссов между абсолютной консистентностью данных и пропускной способностью (Throughput) системы. Идеальная изоляция означает нулевой параллелизм, что неприемлемо для реального бизнеса.

Если бы базы данных выполняли все запросы строго последовательно, один за другим, понятие конкурентного доступа нам бы просто не понадобилось. Никаких конфликтов, никаких гонок за ресурсами. Система была бы математически идеальна и абсолютно безопасна.

Проблема в физике: пока процессор ждёт ответа от медленного диска для транзакции А, он простаивает. В последовательной модели СУБД умерла бы на нагрузке в сотню запросов в секунду. Чтобы утилизировать CPU и I/O дисков, мы обязаны чередовать инструкции разных транзакций.

Мы запускаем их параллельно, разрешая им делить общее состояние в оперативной памяти. Как только две транзакции получают доступ к одной области памяти, начинаются проблемы. Наша главная инженерная задача — разрешить им чередоваться так, чтобы итоговый результат оставался таким же корректным, как если бы они выполнялись строго по очереди.

Computer Science предлагает нам **три фундаментальных способа** справиться с хаосом параллельного доступа. Каждая современная СУБД выбирает один из этих путей.

1. Пессимистичный подход (Lock-based/2PL) — «Не доверяй никому. Блокируй заранее».

- Транзакция, желающая прочитать или изменить строку вешает на неё жёсткий замок. Все остальные выстраиваются в очередь и ждут.
- Гарантирует безопасность, но убивает производительность на чтении. Очереди растут лавинообразно, а разработчикам приходится бороться с взаимными блокировками (Deadlocks).

2. Оптимистичный подход (Optimistic Concurrency Control — OCC) — «Делай, что хочешь в своей песочнице, проверим конфликты перед коммитом».

- Транзакция работает с локальными копиями данных без всяких блокировок. В момент выполнения COMMIT система проверяет: не изменил ли кто-то оригинальные данные, пока мы работали? Если да, транзакция откатывается и перезапускается.
- Идеально для систем с преобладанием чтения. Но при высокой конкуренции за запись (много писателей на одну строку) система тратит все ресурсы на бесконечные откаты.

3. Многоверсионность (Multi-Version Concurrency Control — MVCC) — «У каждого своя версия истории».

- Индустриальный стандарт для тяжеловесов (PostgreSQL, Oracle). База никогда не изменяет данные «на месте». Операция обновления физически создаёт новую версию строки рядом со старой.
- Читатели читают старую версию, писатели создают новую. Читатели никогда не блокируют писателей, а писатели не блокируют читателей.
- Мы размениваем время на дисковое пространство. Старые версии строк копят как мусор, требуя агрессивной работы систем сборки мусора (**Garbage Collection, VACUUM**), иначе база захлебнётся от переполнения диска и деградации I/O.

Прежде чем проектировать механизмы защиты, инженер обязан классифицировать саму угрозу. Исторически фундаментом для этого служит стандарт SQL-92. Он был написан в эпоху, когда индустрия мыслила преимущественно категориями пессимистичных табличных и строчных блокировок. Этот стандарт выделяет три базовые аномалии конкурентного доступа и жёстко привязывает к ним четыре уровня изоляции транзакций.

Первая классическая проблема — это грязное чтение (**Dirty Read**). Представь исходное состояние базы данных: базовая зарплата сотрудника равна 900. Первая транзакция обновляет это значение до 1000, но еще не фиксирует изменения (не делает **COMMIT**). Вторая транзакция параллельно считывает это новое, «грязное» значение (1000) и на его основе внутри бизнес-логики приложения рассчитывает 10-процентную премию. Затем первая транзакция по какой-то причине сталкивается с ошибкой и откатывается (**ROLLBACK**). Зарплата в базе возвращается к исходным 900. В результате вторая транзакция начисляет премию на основе данных, которых официально никогда не существовало в системе. Сразу следует отметить важную деталь: в архитектуре PostgreSQL встретить грязное чтение физически невозможно по умолчанию. Многоверсионный движок просто не умеет читать незафиксированные изменения, однако понимать суть этой концепции необходимо для работы с другими СУБД.

Вторая и третья классические аномалии часто вызывают путаницу у начинающих специалистов, хотя разница между ними фундаментальна и определяет выбор уровня изоляции. Неповторяющееся чтение (**Non-repeatable Read**) возникает на уровне конкретных физических строк. Ты запрашиваешь конкретную запись (например, **SELECT salary ... WHERE id = 1**), затем параллельная транзакция обновляет эту же самую строку и фиксирует свои изменения. При повторном чтении по тому же **id** в рамках твоей транзакции ты обнаружишь, что данные изменились.

Фантомное чтение (Phantom Read) имеет совершенно иную природу. Здесь ты работаешь не с конкретным кортежем, а с математическим множеством, и нагляднее всего это проявляется при использовании агрегатных функций. Ты запрашиваешь количество активных администраторов (**SELECT count(id) ... WHERE role = 'admin'**) и получаешь число 10. В это же время конкурентная транзакция вставляет новую строку и назначает одиннадцатому пользователю права администратора. Твой повторный запрос с тем же предикатом вернёт уже число 11. Сами строки, которые читались изначально, не изменились, но изменилось само множество, удовлетворяющее условию выборки.

Опирайтесь исключительно на стандарт SQL-92 в современных высоконагруженных проектах — гарантированно получить разрушение данных. В 1995 году группа исследователей опубликовала разгромную научную работу с критикой этого стандарта, математически доказав, что трёх аномалий недостаточно. Стандарт оказался абсолютно слеп к феноменам, которые массово возникают в новых системах, построенных на принципах многоверсионности.

Одной из таких пропущенных проблем является потерянное обновление. Вспомни классический пример с гонкой за последним билетом на концерт. Два пользователя одновременно запрашивают доступное количество билетов и оба видят единицу. Оба приложения решают, что могут оформить покупку, вычитают единицу у себя в памяти и отправляют запрос на обновление счётчика до нуля. Вторая транзакция просто затирает результат первой. В итоге система продаёт один физический билет двум разным людям, а деньги списываются у обоих. Это классическая проблема цикла чтения и последующей модификации, от которой стандартные уровни изоляции не защищают, требуя ручного вмешательства в код.

Ещё более коварной и трудноуловимой аномалией является перекося запись. Представь больницу, где по строгим правилам на дежурстве должен оставаться минимум один врач. Два хирурга, Алиса и Боб, параллельно хотят уйти домой. Алиса проверяет базу данных, видит, что Боб числится на смене, и со спокойной совестью обновляет свою запись, ставя статус отсутствия. Боб делает ровно то же самое: видит в базе Алису и уходит. Обе транзакции успешно фиксируются базой. Физического конфликта на уровне отдельных строк не было, ведь каждый обновлял только свою собственную запись. Однако глобальное бизнес-правило было нарушено, и больница осталась без врачей. От перекося записи не спасает даже строгий уровень **Repeatable Read**.

Сопоставляя теоретическую матрицу изоляции с суровой физикой PostgreSQL, инженеру нужно помнить несколько ключевых архитектурных особенностей. Запрашивая в коде уровень **Read Uncommitted**, ты фактически всегда получаешь **Read Committed**, так как база игнорирует попытки заставить её отключить механизм снимков. Уровень изоляции по умолчанию, **Read Committed**, защищает тебя только от грязного чтения, создавая новый снимок реальности перед каждым твоим запросом.

Если ты повышаешь уровень до **Repeatable Read**, происходит интересное отклонение от стандарта: PostgreSQL обеспечивает гораздо более строгую изоляцию. В этом режиме база полностью защищает тебя не только от неповторяющегося чтения, но и от появления фантомов, так как снимок времени замораживается один раз на весь период жизни транзакции. Однако для абсолютной математической гарантии целостности и предотвращения сложных логических конфликтов вроде перекося записи разработчик обязан использовать высший уровень изоляции — **Serializable**.

Переходя от теоретических стандартов к физической реализации, мы должны полностью перестроить своё мышление. В ядре PostgreSQL заложена парадигма многоверсионности, главным правилом которой является то, что читатели не блокируют писателей, а писатели не блокируют читателей. Чтобы достичь этого, движок разменивает процессорное время на дисковое пространство. В PostgreSQL операция обновления данных никогда не изменяет информацию на своем исходном месте. Вместо этого физический **UPDATE** представляет собой комбинацию из пометки старой строки как удалённой и вставки совершенно новой версии этой строки в базу данных. Каждая такая версия кортежа является иммутабельным слепком данных в конкретный момент времени.

За эту магию отсутствия блокировок каждая строка несёт ощутимые накладные расходы в виде служебного заголовка. Для управления видимостью версий PostgreSQL добавляет к каждой строке скрытые системные поля, важнейшими из которых являются идентификаторы транзакций **xmin** и **xmax**. Поле **xmin** можно воспринимать как паспорт строки, в который записывается номер транзакции, породившей эту версию. Поле **xmax** выполняет роль надгробия, в которое транзакция, удаляющая или обновляющая эту строку, вписывает свой идентификатор. Если поле **xmax** пусто, это означает, что данная версия строки всё ещё жива и актуальна.

Однако само по себе знание номера транзакции не даёт базе ответа на вопрос, завершила ли эта транзакция свою работу успешно. Для хранения глобального статуса всех транзакций используется компактная структура под названием **CLOG**, или **журнал статусов**. В этом массиве хранятся состояния вроде «в процессе», «зафиксирована» или «отменена». Но если бы база данных при каждом чтении миллионов строк обращалась к глобальному журналу, производительность системы была бы уничтожена. Для решения этой проблемы инженеры придумали оптимизацию с использованием битов-подсказок. Первая же транзакция, прочитавшая строку и узнавшая её статус из CLOG, кеширует этот статус прямо в заголовок самого кортежа. Из-за этого возникает интересный архитектурный парадокс: обычная операция чтения может вызывать изменение страниц в оперативной памяти и последующую запись на диск.

Даже если строка имеет статус успешной фиксации, этого недостаточно для обеспечения изоляции. Базе данных необходимо знать, была ли эта транзакция зафиксирована до момента старта твоего запроса, или это изменение произошло прямо во время твоего чтения. Для реализации этой логики PostgreSQL использует механизм транзакционных снимков. Снимок — это моментальная фотография состояния конкурентной среды, фиксирующая старейшую активную транзакцию, новейшую выданную транзакцию и массив идентификаторов всех транзакций, находящихся в процессе работы на момент создания снимка. Имея эту информацию для каждой версии строки, система математически вычисляет правила видимости. Если номер транзакции-создателя попадает в массив активных конкурентов или относится к будущему относительно нашего снимка, база сделает вывод, что эта строка невидима для текущего запроса. Именно этот механизм позволяет тысячам транзакций иметь собственный изолированный взгляд на данные без единой блокировки.

Очевидно, что у этой изящной системы есть тёмная сторона, связанная с управлением памятью. Старые, «мёртвые» версии строк не исчезают из файлов таблиц автоматически, они остаются на диске и занимают место. Накопление такого мусора приводит к бесконечному раздуванию таблиц и индексов. Для борьбы с этой проблемой в PostgreSQL работает специальный фоновый процесс **autovacuum**. Его можно сравнить с роботом-пылесосом, который постоянно сканирует базу данных, находит версии строк, гарантированно не нужные ни одной из активных транзакций, и помечает занимаемое ими пространство как свободное для повторного использования. Понимание и настройка работы этого процесса абсолютно критичны для здоровья и производительности любой базы данных под высокой нагрузкой.

Разбирая многоверсионность, мы неизбежно подходим к тому, как эта внутренняя механика проецируется на прикладной код разработчика. Стандартные уровни изоляции в PostgreSQL — это не просто абстрактные гарантии, а совершенно конкретные правила управления транзакционными снимками. То, что ты пишешь в команде установки уровня изоляции, напрямую диктует движку базы данных, в какой именно момент ему следует сфотографировать реальность.

Начнём с самого слабого уровня изоляции — **Read Uncommitted**. Если ты попытаешься явно включить его в PostgreSQL, система молча проигнорирует твою просьбу и установит следующий, более строгий уровень **Read Committed**. Это не баг и не недоработка интерфейса. Архитектура MVCC физически не способна прочитать незафиксированные изменения, поскольку строки с незавершённым идентификатором создателя отбрасываются математическими правилами видимости снимка. Для реализации грязного чтения разработчикам ядра пришлось бы писать отдельную логику в обход базовых механизмов, чего, естественно, делать не стали. В PostgreSQL грязного чтения не существует по самому дизайну движка.

Уровень по умолчанию, **Read Committed**, работает по принципу постоянно подвижного горизонта. В этом режиме база данных создаёт новую фотографию активных транзакций перед выполнением каждого отдельного оператора внутри твоей транзакции. Для быстрых точечных обновлений (OLTP-нагрузки) это идеальная стратегия, минимизирующая вероятность конфликтов. Однако для сложной аналитики это катастрофа. Если твой финансовый отчёт состоит из нескольких последовательных запросов, каждый из них будет работать с немного отличающейся версией реальности. Между твоими командами конкуренты успеют зафиксировать свои переводы, и ты получишь классическое неповторяющееся чтение, когда дебет перестанет сходиться с кредитом.

Чтобы получить по-настоящему консистентный срез данных, инженеры переключаются на уровень **Repeatable Read**. Здесь механика снимков раскрывается во всей своей мощи: фотография реальности делается ровно один раз при выполнении первой инструкции и замораживается до самого конца транзакции. Именно здесь кроется важнейшее архитектурное отличие PostgreSQL от классического стандарта SQL-92. Благодаря тому, что снимок фиксируется единожды, любая новая строка, вставленная конкурентом, неизбежно получит метку времени из «будущего» относительно твоего снимка и будет математически скрыта от твоих запросов. Таким образом, **Repeatable Read** в PostgreSQL автоматически защищает тебя от фантомного чтения без использования ресурсоёмких блокировок.

Обратной стороной этой временной герметичности является жёсткое правило разрешения конфликтов — побеждает тот, кто первым зафиксировал изменения. Если две транзакции в режиме **Repeatable Read** попытаются обновить одну и ту же строку, вторая не будет поставлена в очередь ожидания. Она получит немедленную ошибку сериализации из-за конкурентного обновления. Главное следствие этого поведения для разработчика: при использовании строгих уровней изоляции код приложения обязан уметь перехватывать такие исключения и автоматически перезапускать упавшие транзакции.

Казалось бы, замороженный снимок решает все проблемы конкурентного доступа, но эта герметичность абсолютно слепа к логическим аномалиям вроде перекоса записи. Вспомним наш пример с дежурными врачами. Оба хирурга, находясь в режиме **Repeatable Read**, увидят один и тот же замороженный снимок, где их коллега всё ещё находится на дежурстве. Поскольку они обновляют разные физические записи (каждый ставит статус «ушёл домой» только себе), база данных не увидит конфликта версий кортежей и успешно зафиксирует обе транзакции. Многоверсионность отлично защищает от конфликтов на уровне конкретных строк, но она не способна валидировать сложные бизнес-правила, опирающиеся на пересекающиеся множества записей.

Для решения проблемы перекося записи без возврата к архаичным глобальным табличным блокировкам, в PostgreSQL был внедрён алгоритм **Serializable Snapshot Isolation**. При выборе максимального уровня изоляции, **Serializable**, база данных начинает неявно отслеживать операции чтения, накладывая лёгкие виртуальные предикатные блокировки. Движок в реальном времени выстраивает в оперативной памяти граф зависимостей между транзакциями. Как только алгоритм обнаруживает опасный цикл — то есть ситуацию, когда одна транзакция прочитала то, что изменила другая, и наоборот — система понимает, что формируется перекося записи. База оптимистично позволяет обеим транзакциям выполнять свою работу, но в момент попытки зафиксировать изменения безжалостно прерывает одну из них. Таким образом достигается истинная математическая консистентность, при которой транзакции сериализуются без тяжёлого физического блокирования друг друга.

Разобравшись с тем, как транзакции уживаются друг с другом в оперативной памяти с помощью многоверсионности, мы обязаны ответить на вопрос: как спасти результаты их работы в случае физического отказа оборудования. Это буква **D** в контракте **ACID** — **долговечность**. Наивным подходом было бы записывать изменения непосредственно в основные файлы таблиц на диске в момент подтверждения транзакции. Однако с точки зрения физики накопителей это катастрофа. База данных оперирует страницами памяти размером по восемь килобайт. Если каждая транзакция будет заставлять головку жёсткого диска совершать хаотичные прыжки для обновления разрозненных страниц, пропускная способность системы рухнет до минимальных значений из-за огромных задержек случайного доступа. Более того, если сервер выключится ровно в середине процесса перезаписи такого блока, файл данных может быть повреждён безвозвратно.

Чтобы разрешить эту дилемму между надёжностью и скоростью, реляционные системы используют **механизм журнала предзаписи**, или **WAL**. Главное архитектурное правило гласит: прежде чем изменять основные файлы данных, система обязана очень быстро зафиксировать намерение об этом изменении в специальном отдельном логе. Физически WAL представляет собой непрерывный файл, в который записи добавляются строго в конец. Для диска любая последовательная запись в конец файла выполняется на порядки быстрее случайного доступа, превращая медленную дисковую операцию в сверхбыструю. Этот журнал работает как бортовой самописец в самолёте: он не анализирует полёт, а просто фиксирует каждое изменение байтов в системе.

Фиксации транзакции выглядит следующим образом. Когда твоё приложение отправляет запрос на обновление, PostgreSQL не трогает файлы на диске, а копирует нужную страницу в общую оперативную память, называемую **shared_buffers**, и изменяет данные прямо там. Эта страница в памяти теперь помечается как «грязная». Одновременно формируется бинарная запись для журнала предзаписи, которая помещается в локальный WAL-буфер. В момент выполнения команды **COMMIT** база данных делает единственную критически важную вещь: она приказывает операционной системе немедленно сбросить этот WAL-буфер на физический накопитель. И только когда контроллер диска аппаратно подтверждает стопроцентную уверенность в успешной записи журнала, база отправляет клиенту ответ об успешном завершении транзакции.

Представь, что ровно через миллисекунду после такого ответа в дата-центре выключают свет. Сервер мгновенно падает, и все «грязные» страницы данных, находившиеся в оперативной памяти, бесследно исчезают. Основные файлы таблиц на диске при этом остались в старом, неконсистентном состоянии. Однако катастрофы не происходит. При следующем запуске PostgreSQL обнаруживает флаг аварийного завершения и инициирует процедуру восстановления. База данных открывает WAL-журнал, находит записи о зафиксированных транзакциях и последовательно проигрывает их заново как видеозапись, применяя изменения напрямую к файлам данных на диске. Система восстанавливается в точности до состояния последнего успешного коммита.

За обновление таблиц отвечает скрытый фоновый процесс под названием **checkpointer**. Он периодически, в фоновом режиме, берёт накопившиеся «грязные» страницы из оперативной памяти и записывает их в основные файлы данных на диске. Как только этот процесс успешно завершается, контрольная точка пройдена. PostgreSQL понимает, что все изменения до этого момента надёжно сохранены в таблицах, а значит, старые файлы WAL-журнала больше не нужны для восстановления и их можно безопасно удалить или перезаписать.